Manipal Institute of Technology, Manipal
Department of Computer Science and Engineering

Mini Project — Phase II

Implementing an Online Shopping Cart System

Udit Asthana
240905310
CSE-D

February – April 2026

Contents

1 Abstract **3**

2 Problem Statement **3**
2.1 Background . . . 3
2.2 Data Requirements . . . 3
2.3 Functional Requirements . . . 4

3 E-R Diagram and Schema Extension **4**
3.1 Original E-R Schema (Exercise 6.21) . . . 4
3.2 Extensions Made . . . 5

4 Relational Schema and Normalisation **6**
4.1 Relational Tables . . . 6
4.2 Normalisation . . . 6

5 DDL — Table Definitions and Constraints **6**

6 SQL Queries **7**
6.1 Basic Queries . . . 7
6.2 Complex Queries . . . 8

7 PL/SQL Procedures, Functions, and Triggers **9**
7.1 Functions . . . 9

7.2 Procedures 9
7.3 Triggers 10

8 Java DB Connectivity **10**

9 Transaction Management **11**

10 Sample Data **12**

11 UI Design **12**
11.1 Books Listing 13
11.2 View Cart 13
11.3 Add to Cart 14
11.4 Checkout 15
11.5 Check Stock 16
11.6 Transaction Demo 17

12 Testing Scenarios **17**

13 Conclusion **17**

References **18**

1 Abstract

This project presents the design and implementation of a web-based Shopping Cart Management System intended to demonstrate practical applications of Database Management System (DBMS) concepts. The system enables users to browse items, add or remove products from a virtual shopping cart, update quantities, and perform a consolidated checkout operation. The primary focus is on database modelling, integrity enforcement, and transaction management rather than building a full-scale commercial e-commerce platform.

The design extends the bookstore Entity–Relationship (E-R) schema of Exercise 6.21 (Silberschatz, *Database System Concepts*, 7th ed.) to incorporate additional entities such as *Cart*, *Order*, *Inventory*, and *Payment*, ensuring proper relationships, normalisation, and constraint enforcement. The relational schema is implemented using Oracle 21c XE with primary keys, foreign keys, check constraints, sequences, triggers, and stored procedures. A Java JDBC layer demonstrates database connectivity and calls to PL/SQL procedures for cart operations, stock verification, and atomic order placement.

2 Problem Statement

2.1 Background

Design and implement a shopping cart system that lets shoppers collect items into a shopping cart and purchase them together. The system is built by extending the bookstore E-R schema of **Exercise 6.21, Chapter 6** (Korth, 6th Edition). Stock availability must be verified at checkout, and out-of-stock situations must be handled gracefully.

2.2 Data Requirements

The system must maintain the following data:

- **Books**: identified by ISBN; storing title, publication year, price, genre, author, and publisher.
- **Authors**: name, address, URL.
- **Publishers**: name, address, phone, URL.
- **Customers**: unique email, name, address, phone, hashed password, registration timestamp.
- **Warehouses**: code, address, phone.
- **Inventory**: per-warehouse stock count for each book.
- **Cart**: links a customer to a set of items; tracks status (ACTIVE, CHECKED_OUT, ABANDONED).
- **Cart Items**: book + quantity within a cart.
- **Orders**: placed from a cart; captures total amount and order status.
- **Order Items**: snapshot of each book's price and quantity at order time.

- **Payments**: method, amount, and status for each order.

2.3 Functional Requirements

1. Browse all available books with author/publisher details.
2. Add, update quantity, and remove books from an active cart.
3. Check total stock availability before and during checkout.
4. Atomically place an order from a cart using a transaction; rollback on failure.
5. Automatically decrement inventory on order placement (trigger).
6. Mark a cart as CHECKED_OUT when an order is placed (trigger).
7. Prevent negative inventory at all times (trigger + constraint).
8. Record payment alongside each order.
9. Support complex reporting: revenue by publisher, demand forecasting, order receipts, and customer analytics.

3 E-R Diagram and Schema Extension

3.1 Original E-R Schema (Exercise 6.21)

The original bookstore schema from Silberschatz et al. contains the following entities and relationships:

- **Entities**: *author* (name, address, URL), *book* (ISBN, title, year, price), *publisher* (name, address, phone, URL), *customer* (email, name, address, phone), *shopping_basket* (basket_id), *warehouse* (code, address, phone).
- **Relationships**: *written_by* (book–author), *published_by* (book–publisher), *basket_of* (basket–customer), *contains* (basket–book, with attribute *number*), *stocks* (warehouse–book, with attribute *number*).

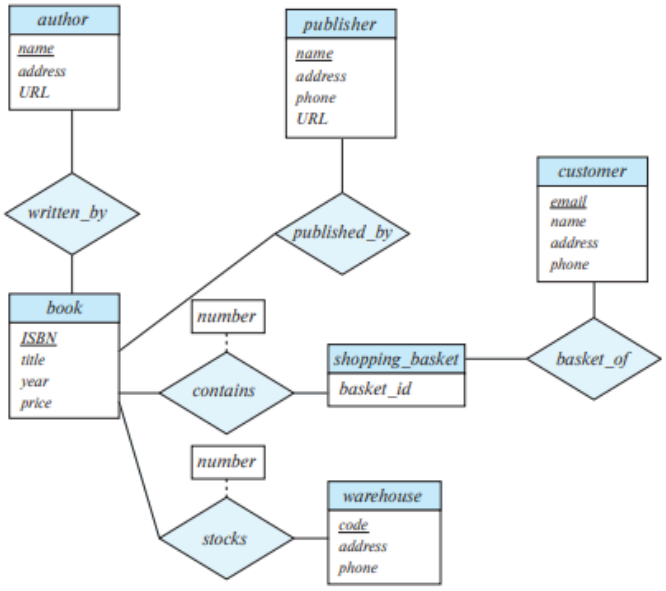

Figure 1: Original E-R schema from Exercise 6.21 (Silberschatz et al.)

3.2 Extensions Made

Table 1 summarises the extensions applied to the original schema.

Table 1: Schema extensions beyond Exercise 6.21

Original	**Change**	**Reason**
book	Added *genre*, *description*	Richer browsing
customer	Added *password_hash*, *created_at*	Auth + analytics
shopping_basket	Renamed to *cart*; added *status*, *created_at*	Lifecycle tracking
contains	Became *CART_ITEMS* table	Proper relation
stocks	Became *INVENTORY* table	Warehouse tracking
—	New: *ORDERS*	Order management
—	New: *ORDER_ITEMS*	Per-order snapshot
—	New: *PAYMENT*	Payment tracking

4 Relational Schema and Normalisation

4.1 Relational Tables

AUTHOR(*name*, *address*, *url*)
PUBLISHER(*name*, *address*, *phone*, *url*)
BOOK(*isbn*, *title*, *year*, *price*, *genre*, *description*, *author_name*, *publisher_name*)
CUSTOMER(*cust_id*, *email*, *name*, *address*, *phone*, *password_hash*, *created_at*)
WAREHOUSE(*code*, *address*, *phone*)
INVENTORY(*warehouse_code*, *isbn*, *quantity*)
CART(*cart_id*, *cust_id*, *status*, *created_at*, *updated_at*)
CART_ITEMS(*cart_id*, *isbn*, *quantity*)
ORDERS(*order_id*, *cust_id*, *cart_id*, *order_date*, *total_amount*, *status*)
ORDER_ITEMS(*order_id*, *isbn*, *quantity*, *price_at_order*)
PAYMENT(*payment_id*, *order_id*, *amount*, *method*, *payment_date*, *status*)

(Underlined attributes are primary keys; foreign key references follow from context.)

4.2 Normalisation

All tables satisfy **Third Normal Form (3NF) / BCNF**:

- **1NF**: All attributes are atomic. No multi-valued columns.
- **2NF**: No partial dependencies exist. In `ORDER_ITEMS` the non-key attribute `price_at_order` depends on the full composite key {order_id, isbn}, not a subset.
- **3NF**: No transitive dependencies. `BOOK` stores `author_name` and `publisher_name` as FKs only — author details live entirely in `AUTHOR`.

A deliberate design choice: `ORDER_ITEMS.price_at_order` snapshots the book price at time of purchase. Although `BOOK.price` exists, this is not a transitive dependency because the historical price is a distinct fact from the current price.

5 DDL — Table Definitions and Constraints

The following listings show representative DDL. Full code is in `schema_setup.sql`.

Listing 1: BOOK and INVENTORY table definitions

```
CREATE TABLE BOOK (
    isbn            VARCHAR2(20)  NOT NULL,
    title           VARCHAR2(255) NOT NULL,
    year            NUMBER(4)     NOT NULL,
    price           NUMBER(8,2)   NOT NULL,
    genre           VARCHAR2(50),
    author_name     VARCHAR2(100) NOT NULL,
    publisher_name  VARCHAR2(100) NOT NULL,
```

```
    CONSTRAINT pk_book        PRIMARY KEY (isbn),
    CONSTRAINT fk_book_author FOREIGN KEY (author_name)
                              REFERENCES AUTHOR(name),
    CONSTRAINT fk_book_pub    FOREIGN KEY (publisher_name)
                              REFERENCES PUBLISHER(name),
    CONSTRAINT chk_book_price CHECK (price > 0),
    CONSTRAINT chk_book_year  CHECK (year BETWEEN 1000 AND 2100)
);

CREATE TABLE INVENTORY (
    warehouse_code VARCHAR2(20) NOT NULL,
    isbn           VARCHAR2(20) NOT NULL,
    quantity       NUMBER(6)    DEFAULT 0 NOT NULL,
    CONSTRAINT pk_inventory     PRIMARY KEY (warehouse_code, isbn),
    CONSTRAINT fk_inv_warehouse FOREIGN KEY (warehouse_code)
                                REFERENCES WAREHOUSE(code),
    CONSTRAINT fk_inv_book      FOREIGN KEY (isbn)
                                REFERENCES BOOK(isbn),
    CONSTRAINT chk_inv_qty      CHECK (quantity >= 0)
);
```

Listing 2: CART and ORDERS with status constraints

```
CREATE TABLE CART (
    cart_id    NUMBER       DEFAULT SEQ_CART.NEXTVAL NOT NULL,
    cust_id    NUMBER       NOT NULL,
    status     VARCHAR2(20) DEFAULT 'ACTIVE' NOT NULL,
    created_at DATE         DEFAULT SYSDATE,
    CONSTRAINT pk_cart       PRIMARY KEY (cart_id),
    CONSTRAINT fk_cart_cust  FOREIGN KEY (cust_id)
                             REFERENCES CUSTOMER(cust_id),
    CONSTRAINT chk_cart_status
        CHECK (status IN ('ACTIVE','CHECKED_OUT','ABANDONED'))
);
```

6 SQL Queries

6.1 Basic Queries

Listing 3: Q1: All books with author and publisher

```
SELECT b.isbn, b.title, b.year, b.price,
       a.name AS author, p.name AS publisher
FROM   BOOK b
JOIN   AUTHOR    a ON a.name = b.author_name
JOIN   PUBLISHER p ON p.name = b.publisher_name
ORDER  BY b.title;
```

Listing 4: Q2: Active cart contents for a customer

```
SELECT b.title, b.price, ci.quantity,
```

```
        (b.price * ci.quantity) AS line_total
FROM    CART c
JOIN    CART_ITEMS ci ON ci.cart_id = c.cart_id
JOIN    BOOK b        ON b.isbn     = ci.isbn
WHERE   c.cust_id = 1 AND c.status = 'ACTIVE';
```

Listing 5: Q3: Total stock per book across all warehouses

```
SELECT b.title, SUM(i.quantity) AS total_stock
FROM   INVENTORY i
JOIN   BOOK b ON b.isbn = i.isbn
GROUP  BY b.title
ORDER  BY total_stock DESC;
```

6.2 Complex Queries

Listing 6: Q9: Stock availability check for cart items

```
SELECT  b.title,
        ci.quantity              AS qty_wanted,
        NVL(SUM(i.quantity),0)   AS total_available,
        CASE
            WHEN NVL(SUM(i.quantity),0) >= ci.quantity THEN 'IN
               STOCK'
            WHEN NVL(SUM(i.quantity),0) > 0            THEN
               'PARTIAL'
            ELSE                                            'OUT OF
               STOCK'
        END AS availability
FROM    CART_ITEMS ci
JOIN    BOOK       b  ON b.isbn = ci.isbn
LEFT JOIN INVENTORY i ON i.isbn = ci.isbn
WHERE   ci.cart_id = 1
GROUP   BY b.title, ci.quantity;
```

Listing 7: Q8: Active cart summary with totals

```
SELECT  cu.name AS customer, c.cart_id,
        COUNT(ci.isbn)             AS distinct_titles,
        SUM(ci.quantity)           AS total_books,
        SUM(b.price * ci.quantity) AS cart_value
FROM    CART c
JOIN    CUSTOMER   cu ON cu.cust_id = c.cust_id
JOIN    CART_ITEMS ci ON ci.cart_id = c.cart_id
JOIN    BOOK       b  ON b.isbn     = ci.isbn
WHERE   c.status = 'ACTIVE'
GROUP   BY cu.name, c.cart_id
ORDER   BY cart_value DESC;
```

Listing 8: Q11: Revenue per publisher (complex aggregation)

```
SELECT  p.name AS publisher,
```

```
        COUNT(DISTINCT o.order_id)           AS total_orders,
        SUM(oi.quantity)                     AS books_sold,
        SUM(oi.price_at_order * oi.quantity) AS revenue
FROM    ORDER_ITEMS oi
JOIN    BOOK        b  ON b.isbn     = oi.isbn
JOIN    PUBLISHER   p  ON p.name     = b.publisher_name
JOIN    ORDERS      o  ON o.order_id = oi.order_id
WHERE   o.status IN ('PLACED','SHIPPED','DELIVERED')
GROUP   BY p.name
ORDER   BY revenue DESC;
```

7 PL/SQL Procedures, Functions, and Triggers

7.1 Functions

Listing 9: get_total_stock: total stock across all warehouses

```
CREATE OR REPLACE FUNCTION get_total_stock(p_isbn IN VARCHAR2)
RETURN NUMBER IS
    v_total NUMBER := 0;
BEGIN
    SELECT NVL(SUM(quantity), 0) INTO v_total
    FROM   INVENTORY WHERE isbn = p_isbn;
    RETURN v_total;
END get_total_stock;
```

7.2 Procedures

Listing 10: add_to_cart: add a book to a customer's active cart

```
CREATE OR REPLACE PROCEDURE add_to_cart(
    p_cust_id  IN NUMBER,
    p_isbn     IN VARCHAR2,
    p_quantity IN NUMBER DEFAULT 1
) AS
    v_cart_id NUMBER;
    v_stock   NUMBER;
BEGIN
    v_stock := get_total_stock(p_isbn);
    IF v_stock < p_quantity THEN
        RAISE_APPLICATION_ERROR(-20010,
            'Insufficient stock. Available: ' || v_stock);
    END IF;
    -- get or create active cart, then upsert cart item
    ...
    COMMIT;
END add_to_cart;
```

Listing 11: checkout: atomic order placement with rollback

```oracle
CREATE OR REPLACE PROCEDURE checkout(
    p_cust_id        IN  NUMBER,
    p_payment_method IN  VARCHAR2,
    p_order_id       OUT NUMBER
) AS BEGIN
    -- 1. Validate cart not empty
    -- 2. Check all items in stock
    -- 3. INSERT into ORDERS
    -- 4. INSERT into ORDER_ITEMS  (triggers fire here)
    -- 5. INSERT into PAYMENT
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END checkout;
```

7.3 Triggers

Table 2: Summary of triggers

Trigger	Event	Action
trg_decrement_stock	AFTER INSERT on ORDER_ITEMS	Decrement INVENTORY
trg_checkout_cart	AFTER INSERT on ORDERS	Mark cart CHECKED_OUT
trg_cart_updated	AFTER DML on CART_ITEMS	Update cart timestamp
trg_prevent_neg_stock	BEFORE UPDATE on INVENTORY	Reject negative qty
trg_lock_checked_out	BEFORE DML on CART_ITEMS	Reject non-ACTIVE cart

Listing 12: trg_decrement_stock trigger

```oracle
CREATE OR REPLACE TRIGGER trg_decrement_stock
AFTER INSERT ON ORDER_ITEMS
FOR EACH ROW
BEGIN
    UPDATE INVENTORY
    SET    quantity = quantity - :NEW.quantity
    WHERE  isbn = :NEW.isbn;
END;
```

8 Java DB Connectivity

The file `ShoppingCartDB.java` demonstrates Oracle JDBC connectivity using the Type-4 thin driver (`ojdbc11.jar`).

Listing 13: JDBC connection setup

```java
private static final String URL =
    "jdbc:oracle:thin:@localhost:1521/XEPDB1";

public static Connection getConnection() throws SQLException {
```

```
    return DriverManager.getConnection(URL, USER, PASSWORD);
}
```

Listing 14: Calling the checkout PL/SQL procedure

```
String sql = "BEGIN checkout(?, ?, ?); END;";
CallableStatement cs = con.prepareCall(sql);
cs.setInt(1, custId);
cs.setString(2, paymentMethod);
cs.registerOutParameter(3, Types.NUMERIC); // OUT p_order_id
cs.execute();
int orderId = cs.getInt(3);
```

Key JDBC patterns demonstrated:

- `Statement` for simple SELECT queries.
- `PreparedStatement` for parameterised queries (prevents SQL injection).
- `CallableStatement` for PL/SQL procedure and function calls.
- Manual `con.setAutoCommit(false)` with explicit `COMMIT`/`ROLLBACK`.

9 Transaction Management

The checkout operation is the most critical transaction in the system. It follows an all-or-nothing strategy:

1. Validate that the cart contains at least one item.
2. For every cart item, verify that total warehouse stock $\geq$ quantity requested. If any item fails, raise an error before any `INSERT`.
3. Insert one row into `ORDERS`.
4. Insert rows into `ORDER_ITEMS`; triggers fire automatically to decrement `INVENTORY` and mark the cart `CHECKED_OUT`.
5. Insert one row into `PAYMENT`.
6. `COMMIT` — all five steps succeed atomically.
7. `ROLLBACK` in the `WHEN OTHERS` exception handler if any step fails.

This guarantees that partial orders (e.g., payment recorded but inventory not decremented) can never occur.

10 Sample Data

Table 3: Sample books in the system

ISBN	Title	Year	Price (Rs.)
978-0-07-352332-3	Database System Concepts	2019	899
978-0-13-468599-1	Operating System Concepts	2018	799
978-0-13-235088-4	Clean Code	2008	649
978-0-13-212227-1	Computer Networks	2021	749
978-0-06-231609-7	Sapiens	2015	499
978-0-06-244558-9	Homo Deus	2017	549

Table 4: Warehouses and total inventory

Code	Location	Books Stocked
WH-MUM	Andheri East, Mumbai	3 titles
WH-BLR	Whitefield, Bengaluru	3 titles
WH-DEL	Okhla Phase II, New Delhi	3 titles

11 UI Design

The application is built using **JavaFX** with a dark-themed dashboard interface. It connects to Oracle 21c XE via JDBC and exposes all six core operations as tabs. A persistent console panel at the bottom logs every DB operation in real time, and a status indicator in the header confirms the active Oracle connection (`XEPDB1`).

11.1 Books Listing

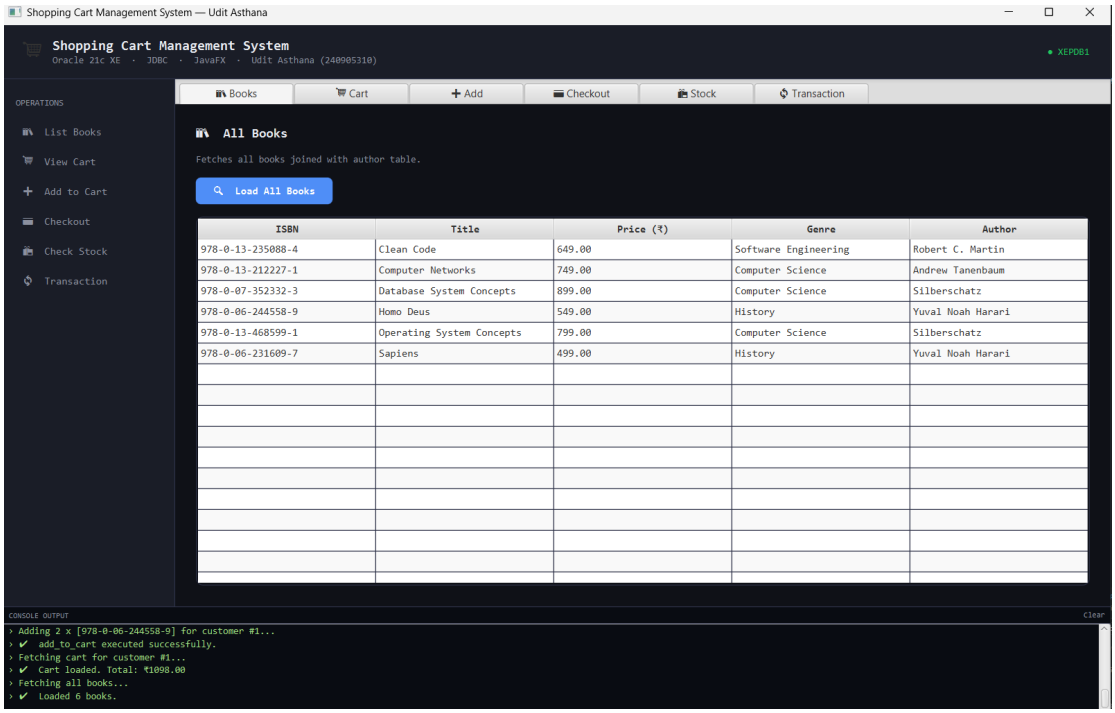

Figure 2: Books tab — all 6 books fetched via `SELECT ... JOIN AUTHOR`. Console shows `Loaded 6 books.`

11.2 View Cart

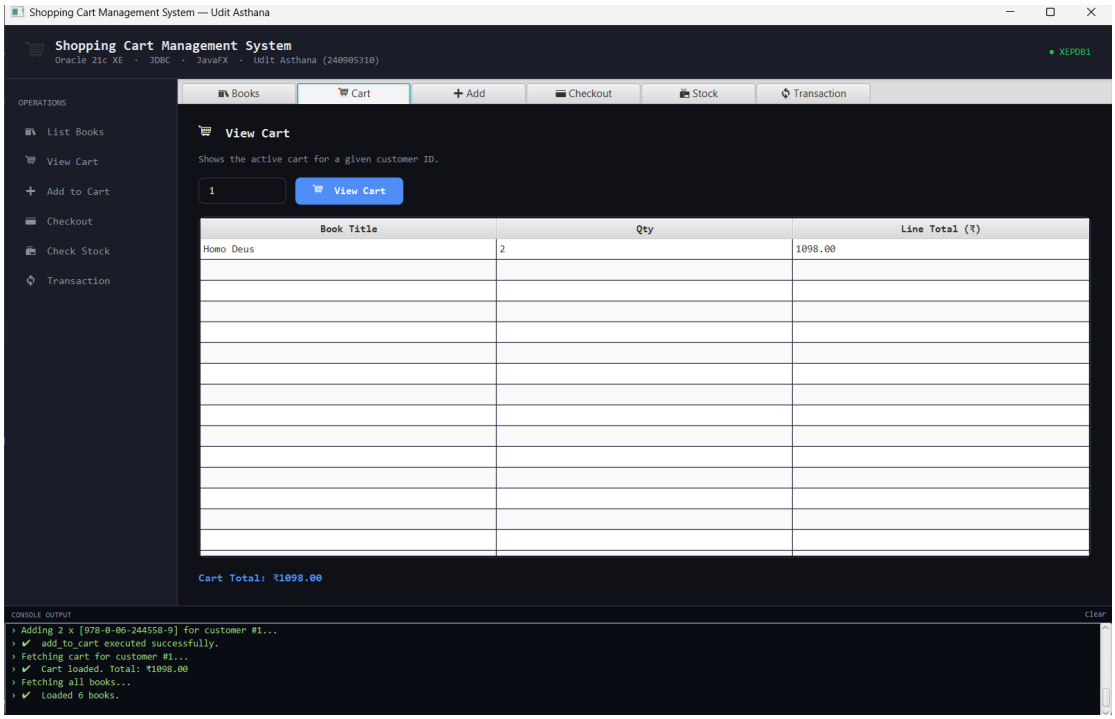

Figure 3: Cart tab — active cart for Customer #1 showing *Homo Deus* (qty 2, line total 1098.00). Uses `PreparedStatement` with customer ID as parameter.

11.3 Add to Cart

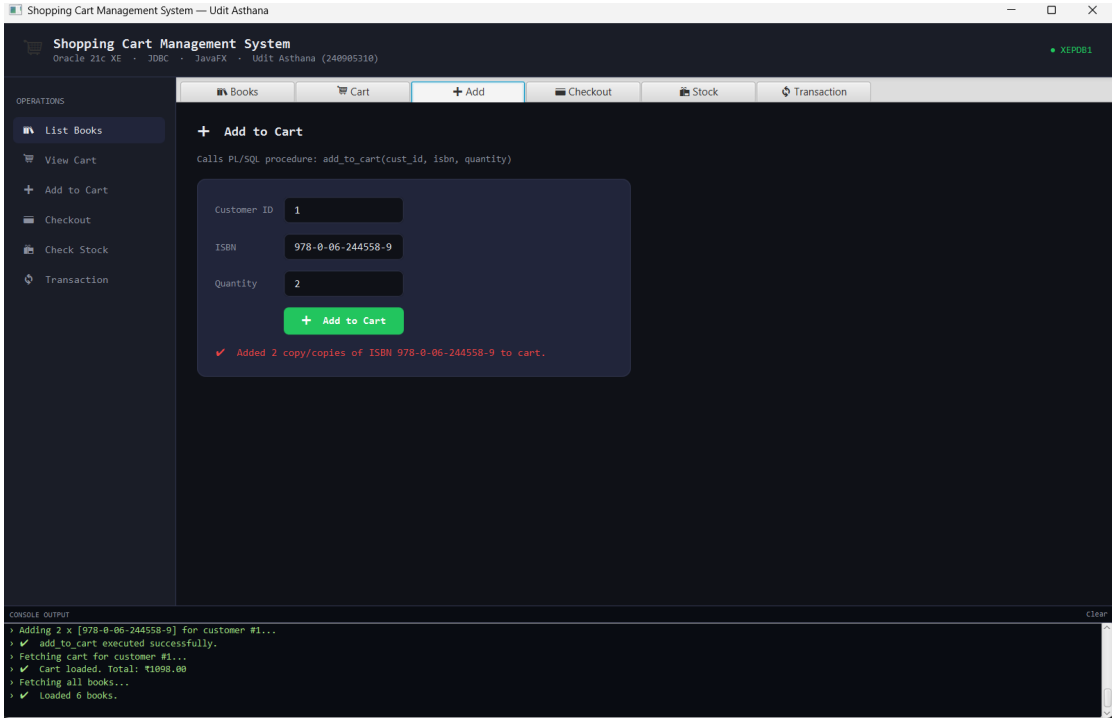

Figure 4: Add to Cart tab — Customer #1 adds 2 copies of ISBN `978-0-06-244558-9`. Calls PL/SQL procedure `add_to_cart(cust_id, isbn, quantity)` via `CallableStatement`.

11.4 Checkout

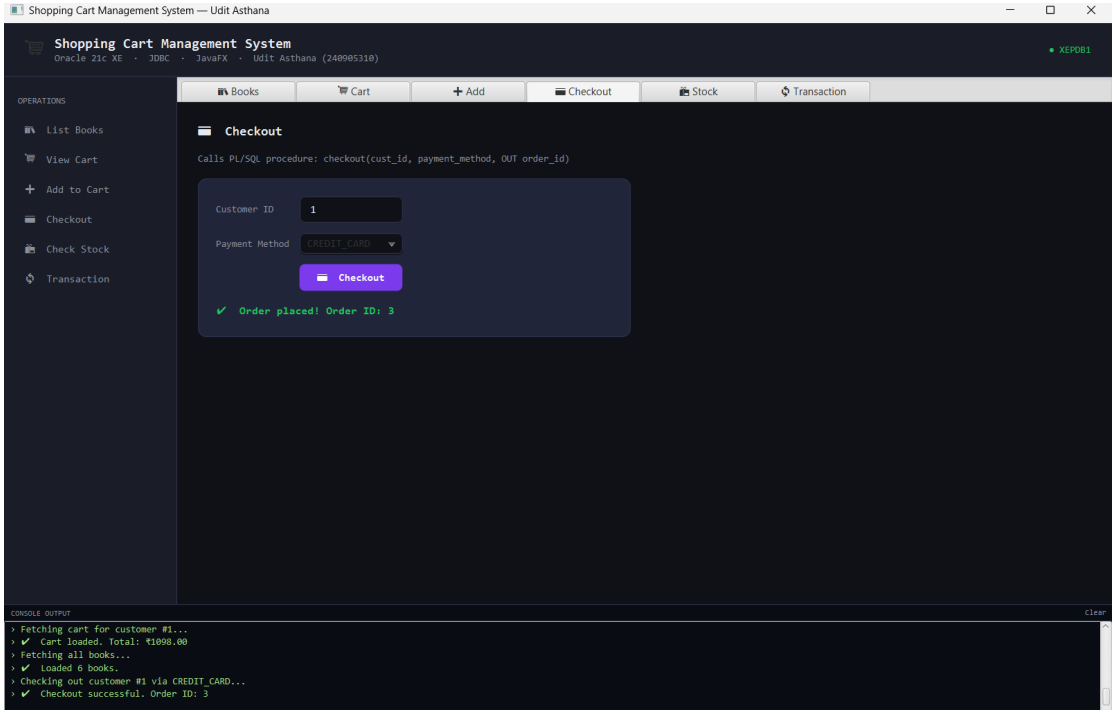

Figure 5: Checkout tab — Customer #1 checks out via `CREDIT_CARD`. Order ID 3 returned from the `checkout` PL/SQL procedure OUT parameter. Console confirms `Checkout successful.`

11.5 Check Stock

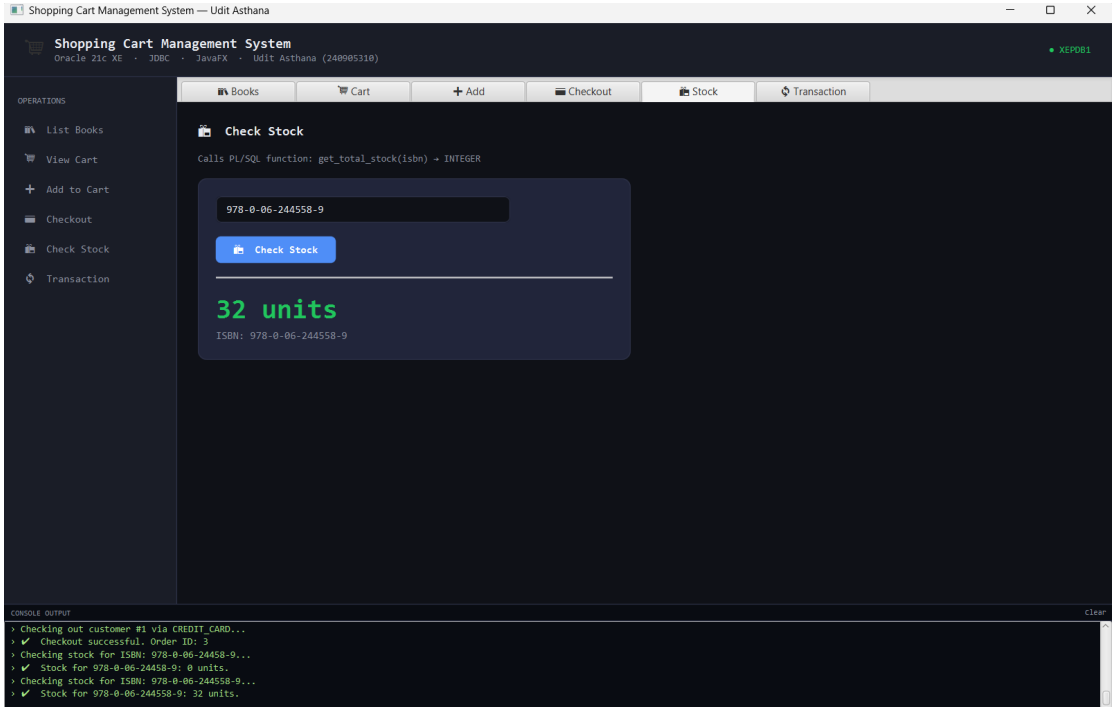

Figure 6: Stock tab — calls PL/SQL function `get_total_stock(isbn)` and displays result. ISBN `978-0-06-244558-9` shows 32 units remaining after the checkout above decremented inventory via trigger.

11.6 Transaction Demo

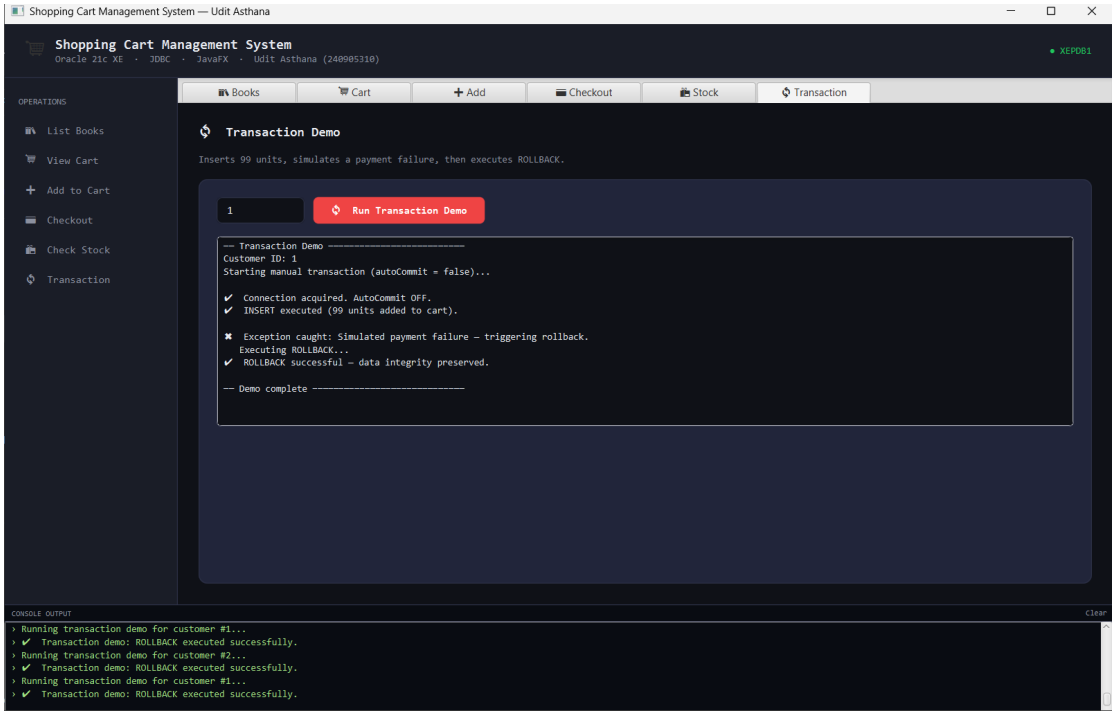

Figure 7: Transaction Demo tab — inserts 99 units into cart, simulates a payment failure, then executes `ROLLBACK`. Demonstrates `con.setAutoCommit(false)` with explicit rollback on exception. Data integrity is preserved.

12 Testing Scenarios

Table 5: Test cases and expected outcomes

Test Case	Input	Expected Result
Add to cart (valid)	ISBN + qty in stock	Item added, cart updated
Add to cart (out of stock)	qty > total stock	ORA-20010 raised
Checkout (valid)	Active cart, items available	Order placed, stock decremented
Checkout (empty cart)	Cart with 0 items	ORA-20030 raised
Checkout (partial stock)	One item insufficient	ORA-20031 raised, rollback
Modify checked-out cart	DML on cart_id=2	ORA-20060 raised (trigger)
Negative stock update	SET quantity = -1	ORA-20050 raised (trigger)
Duplicate email registration	Same email twice	ORA-00001 (unique constraint)

13 Conclusion

This project successfully demonstrates the design and implementation of a Shopping Cart Management System grounded in rigorous DBMS principles. Starting from the Exercise 6.21 bookstore E-R schema, the system was extended to support a complete purchase lifecycle — from cart creation through checkout to payment recording.

Key outcomes achieved:

- A fully normalised (3NF) relational schema with 11 tables.
- 15 SQL queries spanning basic retrieval to complex multi-table aggregation.
- 4 PL/SQL stored procedures and 2 functions encapsulating core business logic.
- 5 triggers enforcing referential and business-rule integrity automatically.
- An atomic, rollback-safe checkout transaction preventing partial order anomalies.
- A Java JDBC layer demonstrating `PreparedStatement`, `CallableStatement`, and manual transaction control.

References

1. Silberschatz, A., Korth, H.F., Sudarshan, S. (2019). *Database System Concepts*, 7th edition. McGraw-Hill.
2. Oracle Corporation. (2021). *Oracle Database 21c SQL Language Reference*. `https://docs.oracle.com/en/database/oracle/oracle-database/21/`
3. Oracle Corporation. (2021). *Oracle Database PL/SQL Language Reference*.
4. Oracle Corporation. (2021). *JDBC Developer's Guide*.